

Semana 8 — HealthComponent, AnimationTree, BlendSpace e AnimationPlayer

Semana 8

HealthComponent, AnimationTree, BlendSpace e AnimationPlayer

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade III — Resolver Problemas (Semanas 8–11)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender vida/dano como problema universal resolvido por composição (Component), não por herança
- Construir `HealthComponent` (vida atual/máxima, `apply_damage`, `reiniciar()`, sinal `died`) e o fluxo de morte/respawn do Player (`GameManager.player_morreu()` → respawn no `Checkpoint` ou `GameOver`)
- Fundamentar AnimationTree/AnimationNodeStateMachine (transição) e BlendSpace1D/2D + faixas do AnimationPlayer (combinação/sobreposição)
- Propor e implementar, com autonomia crescente, uma animação contextual conectada a um evento real de gameplay

Semana 8 — HealthComponent, AnimationTree, BlendSpace e AnimationPlayer

ENCONTRO 1

HealthComponent e State Machine de Locomoção

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 7 (integração final do Módulo 2, Code Review e Playtest) (15 min)
- Demonstração e laboratório: substituição da `CapsuleMesh` do Player pelo modelo do Kenney Mini Characters (15 min)
- Introdução: mudança de metodologia para Challenge Based Learning; o problema do estado de vida/dano compartilhado (15 min)
- Demonstração: construção do `HealthComponent` aplicado ao Player (20 min)
- Demonstração: fluxo de morte/respawn — `died` → `GameManager.player_morreu()` → `spawn_player()` / `GameOver` (15 min)
- Demonstração: AnimationTree/AnimationNodeStateMachine e construção guiada da State Machine (25 min)
- Laboratório: cada grupo aplica `HealthComponent`, o fluxo de morte e ajusta a State Machine (25⁴ min)

Pré-requisito — Do Placeholder ao Modelo Animado

- Desde a Semana 2, o Player usa uma `CapsuleMesh` (placeholder de graybox) como Node `Mesh`
- Hoje ela é substituída pelo modelo do **Kenney Mini Characters**, importado em `assets/characters/` desde a Semana 1
- O modelo já traz um `AnimationPlayer` com as animações idle, walking e running — base direta da State Machine desta aula

Semana 8 — HealthComponent, AnimationTree, BlendSpace e AnimationPlayer



Onde deve viver o estado de vida de um personagem, se Player e Enemy (Semana 11) precisam do mesmo comportamento?

Pense no que `InteractionComponent` e `SaveComponent` já resolveram desde a Semana 5.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema: Estado de Vida Compartilhado, Sem Duplicação

- Vida e dano são um exemplo canônico de estado que precisa ser compartilhado por múltiplos tipos de personagem (Player e, na Semana 11, Enemy)
- A disciplina já ensina, desde a Semana 5, que a resposta do Godot é composição via Component — um Node filho com responsabilidade isolada
- O `HealthComponent` aplica o mesmo princípio: qualquer Scene que o possua como filho ganha vida, dano e um sinal de morte

HealthComponent : Vida, Dano e Morte como Component

- Node customizado, seguindo o mesmo padrão de InteractionComponent (Semana 5) e SaveComponent (Semana 7)
- Implementado via Orchestrator ou GDScript, à escolha do professor/grupo
- Propriedades: vida atual e vida máxima
- Métodos públicos: apply_damage(quantidade) , reiniciar() (restaura o máximo); sinal died emitido quando a vida chega a zero

Fluxo de Morte/Respawn do Player

- O Player conecta `HealthComponent.died` a um handler que chama `GameManager.player_morreu()` — wiring no Player, **regra no GameManager** (mesmo padrão do handler de coleta)
- `player_morreu()`: `SaveManager.mortes += 1`; se ainda há tentativas → `HealthComponent.reiniciar()` + `spawn_player()` (respawn no último `Checkpoint`, DC-06); senão → Scene `GameOver`
- Vida sempre volta ao máximo no respawn → **vida do Player não é persistida**; só `mortes: int` entra no `SaveData`
- `GameOver` (Control + CanvasLayer): pausa o jogo; "Reiniciar" apaga o save e volta ao `PlayerStart`
- `LIMITE_TENTATIVAS` é um placeholder no `GameManager`, ajustável por grupo

Estado de Vida/Dano no Godot × Unity

Godot

- **HealthComponent** : Node customizado (Orchestrator ou GDScript), seguindo a convenção de Component já usada no projeto desde a Semana 5

Unity

- Sem equivalente formal nativo
- Padrão comum: MonoBehaviour próprio anexado ao GameObject, mesma ideia de composição via Component

Demonstração — Construção do HealthComponent

O que será construído (via Orchestrator ou GDScript):

- HealthComponent (Node) com vida atual/máxima, apply_damage e sinal died
- Aplicação do HealthComponent como filho do Player

Por quê: evita lógica de vida/dano duplicada entre Player e Enemy — qualquer Scene que precisar de vida conversa apenas com seu próprio HealthComponent.

Imagem sugerida

Objetivo: mostrar o HealthComponent como Node filho reutilizável.

Enquadramento: duas árvores de cena lado a lado.

Elementos presentes: à esquerda, "Player" com filho "HealthComponent"; à direita, "Enemy" (Semana 11) com o mesmo filho "HealthComponent", destacado com a mesma cor nas duas árvores.

Destaque visual: o HealthComponent idêntico nas duas árvores, reforçando reutilização sem duplicação.

Legenda sugerida: "HealthComponent: o mesmo Component, reutilizado por Player e Enemy, sem herança compartilhada."

Fundamentando AnimationTree: Quem Decide Qual Animação Toca

- Toda engine com um personagem animado enfrenta o mesmo problema: animações isoladas (idle, andar, correr) precisam ser organizadas em um sistema que decide qual toca e como transiciona
- AnimationPlayer guarda as animações; AnimationTree decide qual delas toca e quando, separado da lógica de gameplay
- AnimationNodeStateMachine organiza essa decisão como estados e transições, cada uma com uma condição

AnimationNodeStateMachine: Estados e Transições

- Cada estado da State Machine é uma animação (idle, andar, correr)
- Cada transição depende de uma condição real (ex.: velocidade, input), nunca de uma transição automática sem critério
- Construção guiada de uma State Machine básica de locomoção para o Player

Transição de Animações no Godot × Unity

Godot

- AnimationTree (Node na própria Scene) + AnimationNodeStateMachine
- Integrado à árvore de cena, seguindo o modelo de composição da disciplina

Unity

- Animator Controller (asset `.controller`), vinculado a um Animator Component
- Estados e transições configurados fora da hierarquia de GameObjects

Arquitetura — HealthComponent e AnimationTree como Camadas Independentes

- HealthComponent e AnimationTree resolvem problemas diferentes e não se conhecem diretamente
- Nenhum dos dois altera GameManager, SaveManager, contrato Interactable ou ItemData/Enum das semanas anteriores
- A conexão entre os dois (animação reagindo a dano) só aparece no desafio do Encontro 2

Laboratório — HealthComponent e State Machine de Locomoção

Cada grupo replica, no próprio projeto:

1. `HealthComponent` (Node) com vida atual/máxima, `apply_damage`, `reiniciar()` e sinal `died`, como filho do Player
2. Wiring `died` → `GameManager.player_morreu()`; `player_morreu()` incrementa `SaveManager.mortes`, respawna via `spawn_player()` ou exibe `GameOver` ao atingir `LIMITE_TENTATIVAS`
3. `GameOver.tscn` (Control + CanvasLayer) com "Reiniciar" (apaga save, volta ao `PlayerStart`)
4. AnimationTree com AnimationNodeStateMachine para idle, andar e correr, transições condicionadas a uma variável real de movimento

Boas Práticas — Component e State Machine

- `HealthComponent` enxuto: apenas vida, dano, `reiniciar()` e o sinal `died` — a lógica de game over/respawn vive no `GameManager`, não no Component
- Nomear estados da State Machine de forma clara e consistente com as animações do AnimationPlayer
- Um único `HealthComponent` ativo por personagem, assim como um único `SaveComponent` por projeto
- Testar cada transição isoladamente antes de avançar para a próxima

Fechamento — Encontro 1

- `HealthComponent` funcional aplicado ao Player (vida, `apply_damage`, `reiniciar()`, sinal `died`)
- Fluxo de morte/respawn: Player morre → respawn no último `Checkpoint` com vida cheia; ao esgotar `LIMITE_TENTATIVAS` → `GameOver`
- State Machine básica de locomoção via `AnimationTree`, alternando corretamente entre idle, andar e correr
- `SaveData` ganha `mortes: int`; `GameManager` ganha `player_morreu()` — demais sistemas do Módulo 2 sem alteração
- Próximo passo: `BlendSpace`, faixas do `AnimationPlayer` e o desafio de animação contextual, no Encontro 2

Semana 8 — HealthComponent, AnimationTree, BlendSpace e AnimationPlayer

ENCONTRO 2

BlendSpace, Faixas do AnimationPlayer e Desafio de Animação Contextual

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (HealthComponent , State Machine básica) (15 min)
- Demonstração: fundamentação e configuração de um BlendSpace1D/2D direcional (25 min)
- Demonstração: configuração de uma animação pontual via faixa do AnimationPlayer (20 min)
- Apresentação do desafio: escolha entre BlendSpace ou animação pontual conforme o problema proposto pelo grupo (15 min)
- Laboratório/Desafio: cada grupo propõe e implementa sua animação contextual própria (45 min)
- Feedback e fechamento da semana (15 min)

Semana 8 — HealthComponent, AnimationTree, BlendSpace e AnimationPlayer



Uma reação a dano e uma corrida direcional são o mesmo tipo de problema de animação?

Pense na diferença entre algo que varia continuamente e algo que acontece uma única vez.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema: Misturar Continuamente Versus Sobrepor Pontualmente

- A State Machine resolve transição entre estados discretos (idle → andar → correr)
- Nem toda animação é discreta: locomoção direcional varia continuamente (velocidade e direção)
- Nem toda animação é contínua: reagir a dano, atacar ou responder a uma interação são eventos pontuais, sobrepostos à locomoção de base

BlendSpace1D/2D: Interpolação Multidimensional

- BlendSpace1D interpola ao longo de um único eixo contínuo (ex.: parado até correndo)
- BlendSpace2D interpola em duas dimensões simultaneamente (ex.: direção e velocidade)
- Evita a explosão combinatória de criar uma animação para cada combinação possível de direção e intensidade

Faixas do AnimationPlayer: Animações Pontuais Sobrepostas

- Mecanismo para animações discretas sobrepostas à locomoção de base (reação a dano, gesto de interação, ataque)
- Não substitui a State Machine nem o BlendSpace — soma-se a eles sem quebrar a locomoção
- Critério de escolha: o movimento varia continuamente (BlendSpace) ou é um evento único (faixa do AnimationPlayer)?

Mistura e Sobreposição de Animações no Godot × Unity

Godot

- BlendSpace1D/2D: pontos configurados explicitamente dentro do AnimationTree
- Faixas do AnimationPlayer para sobreposição pontual simples

Unity

- Blend Tree (1D ou 2D, Simple Directional/Freeform) dentro do Animator Controller, por parâmetros (floats)
- Animation Layers com Avatar Masks para sobreposição parcial do esqueleto

Demonstração — BlendSpace Direcional e Faixa Pontual

O que será construído:

- Um BlendSpace1D/2D direcional simples sobre o Player
- Uma animação pontual via faixa do AnimationPlayer (reação a dano, gesto de ataque ou interação)

Por quê: dar a cada grupo os dois mecanismos disponíveis antes de decidir qual deles resolve o desafio proposto.

Imagem sugerida

Objetivo: contrastar BlendSpace (contínuo) e faixa do AnimationPlayer (pontual).

Enquadramento: diagrama de duas colunas.

Elementos presentes: à esquerda, "BlendSpace2D" com eixos de velocidade e direção e um ponto se movendo entre animações; à direita, "Faixa do AnimationPlayer" com uma linha do tempo e um evento pontual sobreposto à animação base.

Destaque visual: o ponto contínuo no BlendSpace versus o marcador único na faixa pontual.

Legenda sugerida: "BlendSpace mistura continuamente; faixas do AnimationPlayer sobrepõem eventos pontuais à locomoção de base."

Arquitetura — Onde a Animação Contextual se Conecta ao Vertical Slice

- A animação contextual deve se conectar a um evento real já existente: sinal `died` /dano do `HealthComponent`, ou contrato `Interactable`
- Nenhuma condição artificial ou tecla de teste isolada — sempre um evento real de gameplay
- A State Machine de locomoção do Encontro 1 permanece intacta; a animação contextual se sobrepõe a ela, não a substitui



Desafio — Animação Contextual Própria

Proponha e implemente uma animação contextual conectada a um evento real do projeto (dano via `HealthComponent`, interação via contrato `Interactable`, ou ataque), escolhendo entre `BlendSpace` ou animação pontual conforme a natureza do problema.

OBJETIVOS

Entrega: animação contextual funcional, conectada a um evento real de gameplay, sem quebrar a State Machine de locomoção do Encontro 1.

Boas Práticas — Animação Contextual

- Perguntar sempre "esse movimento varia continuamente ou é um evento único?" antes de escolher o mecanismo
- Justificar a escolha entre BlendSpace e animação pontual pela natureza do problema, não por preferência arbitrária
- Testar a animação contextual sem quebrar a State Machine de locomoção já construída
- Conectar sempre a um evento real do Vertical Slice, nunca a um teste isolado

Fechamento — Encontro 2

- Animação contextual própria, conectada a um evento real (`HealthComponent` ou contrato `Interactable`)
- Escolha entre BlendSpace e animação pontual justificada pela natureza do problema
- State Machine de locomoção do Encontro 1 preservada, sem retrabalho
- Primeiro desafio da Unidade III sob Challenge Based Learning concluído

Resultado Esperado da Semana

- Modelo do Kenney Mini Characters substituindo a `CapsuleMesh` de placeholder no Player
- `HealthComponent` funcional aplicado ao Player (vida, `apply_damage`, `reiniciar()`, sinal `died`)
- Fluxo de morte/respawn completo: respawn no `Checkpoint`, contador `mortes`, tela `GameOver` ao esgotar as tentativas
- State Machine básica de locomoção via AnimationTree (idle, andar, correr)
- Animação contextual própria — BlendSpace direcional ou animação pontual — conectada a um evento real de gameplay
- Turma domina AnimationTree, AnimationNodeStateMachine, BlendSpace1D/2D e faixas do AnimationPlayer como camadas complementares, relacionando-as ao Animator Controller/Blend Tree da Unity

Checklist da Semana

- [] `CapsuleMesh` de placeholder substituída pelo modelo do Kenney Mini Characters, com `AnimationPlayer` funcional
- [] `HealthComponent` (Node) com vida atual/máxima, `apply_damage`, `reiniciar()` e sinal `died`
- [] `died` do Player → `GameManager.player_morreu()` → respawn no `Checkpoint` (vida cheia, mortes++) ou `GameOver` ao atingir `LIMITE_TENTATIVAS`
- [] `GameOver.tscn` pausa o jogo; "Reiniciar" apaga o save e volta ao `PlayerStart`
- [] State Machine via `AnimationTree` alternando corretamente entre idle, andar e correr
- [] `BlendSpace1D/2D` ou faixa do `AnimationPlayer` configurados na demonstração
- [] Animação contextual própria conectada a um evento real (`HealthComponent` ou `Interactable`)
- [] State Machine de locomoção preservada, sem retrabalho

Próximos Passos — Semana 9

O `HealthComponent` construído nesta semana é consumido diretamente pelo HUD da Semana 9, que passa a exibir em tempo real dados de gameplay já existentes — vida, itens, progresso — via Control nodes e CanvasLayer. A State Machine e as animações contextuais não são retomadas diretamente na Semana 9, mas permanecem no projeto como parte do Vertical Slice, prontas para reaparecer na Semana 11, quando o `HealthComponent` é reutilizado pelo Enemy em um sistema de combate simples.

Leitura recomendada: Godot Docs — Animation; Unity Manual (consulta comparativa) — Animator Controller, Blend Trees.